Reinforcement Learning for Sim-to-Sim Robot Transfer

Leonardo Passafiume (358616) Lucio Baiocchi Maurizio Lanzoni Raiteri
Thomas Kelly

Abstract

This report studies reinforcement learning for continuous control and sim-to-real transfer in the simplified sim-to-sim setting proposed by the project. First, we implemented REINFORCE, REINFORCE with a constant baseline, and an actor-critic policy-gradient variant from scratch on `Hopper-v4`*. Then, we moved to the* `PandaPush-v3` *manipulation task and trained Stable-Baselines3 PPO and SAC agents under source and target dynamics, where the target object is heavier than the source object. The final implementation includes an evaluation for fixed and randomized object mass and friction, a reward-shaping wrapper, Uniform Domain Randomization (UDR) over mass and friction, and an Adaptive Domain Randomization (ADR) wrapper. The main results obtained for the PandaPush-v3 task show that a well tuned SAC policy can transfer quite well under low sim-to-real changes. However, when the sim-to-real gap is marked (hard evaluation setting) vanilla SAC fails and Domain Randomization (DR) are needed to close the gap.*

1. Introduction

Reinforcement learning (RL) formulates control as sequential decision making, where an agent learns a policy by interacting with an environment and maximizing expected return [9]. In robotics this setting is attractive because it can optimize behaviors that are difficult to hand-design, but it is also difficult because interactions are expensive, exploration can be unsafe, and the learned policy can exploit simulator-specific details [4, 5]. A common practical solution is to train in simulation and transfer the policy to the real robot. The central difficulty is the reality gap: the simulator dynamics are never exactly equal to the deployment dynamics [3].

This project follows the course sim-to-real abstraction in a controlled sim-to-sim benchmark. In Part 1, we implement basic policy-gradient algorithms on `Hopper-v4` to study the variance and stability issues behind direct policy optimization. In Part 2, we use `PandaPush-v3`, where the source domain has a light object and the target domain has a heavier object. The target domain plays the role of the real world: it is available for evaluation, but training directly on it is treated as an oracle upper bound rather than as the deployable solution.

The second part uses modern actor-critic algorithms from Stable-Baselines3. PPO constrains policy updates through a clipped surrogate objective [8], while SAC is an off-policy maximum-entropy method that learns stochastic policies and reuses data through a replay buffer [2]. To reduce the transfer gap, we implement domain randomization, which trains over a distribution of simulator parameters instead of a single nominal model [6, 7, 10]. The implemented UDR wrapper samples parameters from fixed ranges, while the ADR wrapper adapts the mass range based on recent success, following the curriculum idea behind automatic domain randomization [1].

2. Methodology

2.1. Part 1: Policy Gradients on Hopper

The first part uses `Hopper-v4`, a continuous-control MuJoCo task. The action is continuous and the policy is stochastic. In `part1/agent.py`, the policy network is a two-hidden-layer MLP with 64 units per layer and `tanh` activations. The actor outputs the mean of a diagonal Gaussian distribution; the standard deviation is a learned parameter passed through `softplus`. All Part 1 experiments use Adam with learning rate 3×10^{-4} and discount factor $\gamma = 0.99$.

REINFORCE. For an episode trajectory, the implementation computes Monte-Carlo returns

$$G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k, \tag{1}$$

and minimizes the negative policy-gradient objective

$$\mathcal{L}_{\mathrm{PG}} = -\sum_t (G_t - b) \log \pi_\theta(a_t|s_t), \tag{2}$$

where b is either zero or a constant baseline.

Actor-Critic Variant. The actor-critic implementation uses the same actor network plus a critic network that predicts $V_\phi(s)$. At each update, the critic is trained by regression to the one-step bootstrapped target

$$y_t = r_t + \gamma V_\phi(s_{t+1})(1 - d_t), \tag{3}$$

where d_t indicates episode termination and the next-state value is detached from the gradient. The actor uses the corresponding advantage estimate

$$A_t = y_t - V_\phi(s_t), \tag{4}$$

which is normalized over the collected episode before computing the policy-gradient loss.

2.2. Part 2: PandaPush Pipeline

The second part uses the `panda-gym` environment. The source object mass is 1.0 kg, while evaluation is performed on two target settings described in the evaluation protocol. The goal position is fixed at the center of the table, while the object initial position is randomized in the xy plane. The training scripts use the dense task reward and an episode horizon of 500 steps.

Training Setting The training was performed trying different hyperparameters, and then choosing the ones that best performed. Table 1 reports the main hyperparameters used for the final SAC and PPO runs.

Hyperparameter	SAC	PPO
Training steps	500k	500k
Policy	`MultiInputPolicy`	`MultiInputPolicy`
Learning rate	$10^{-3}, 3e^{-4}, 10^{-4}$	$10^{-3}, 3e^{-4}, 10^{-4}$
Discount γ	0.95, 0.98, 0.99	0.95, 0.98, 0.99
Batch size	512, 1024, 2048	512, 1024, 2048
Gradient steps / epochs	-1	5
Rollout steps	–	2048
Learning starts	100k	–
Replay buffer size	10^6	–
Target update τ	0.05	–
Entropy coefficient	auto	0.01
GAE λ	–	0.95
Clip range	–	0.2
Policy network	default SB3	[512, 256, 128]

Table 1. Main hyperparameters used for the Part 2 Stable-Baselines3 training runs. Dashes indicate algorithm-specific parameters that are not used.

Reward Shaping. The wrapper `part2/wrappers.py` changes only the scalar training reward, not the dynamics. The shaped reward is

$$r_{\text{train}} = r_{\text{env}} - p + B\mathbf{1}[d(o, g) < \epsilon], \tag{5}$$

where:

- p is a per-step penalty,
- B is a close-goal bonus,
- $d(o, g)$ is the object-goal distance

The values used for the training are $p = 10^{-2}$, $B = 1.0$, and $\epsilon = 0.05$.

Reward shaping was added on top of the dense environment reward to encourage faster goal reaching through a small per-step penalty and an additional bonus when the object is close to the target. Evaluation uses the environment reward directly, so success rate is not computed from the shaped reward.

Domain Randomization. The wrapper `part2/rand_wrapper.py` implements three modes. In `none`, the nominal environment dynamics are kept. In `udr`, the wrapper samples a new mass and friction values at every reset. The current UDR training ranges are:

$$m \sim \mathcal{U}(1, 5), \quad \mu_{\text{obj}}, \mu_{\text{table}} \sim \mathcal{U}(0.5, 1.2), \tag{6}$$

and object spinning friction is sampled from $\mathcal{U}(0, 0.005)$. In `adr`, the wrapper starts from the nominal mass and friction and adapts the sampling ranges using a window of 20 episode outcomes. If the mean success rate is at least 0.70, the mass upper bound expands toward 5 kg and the friction ranges widen toward their global limits. If the mean success rate is at most 0.35, the ranges contract back toward the nominal values. With the default step size, the mass range expands by 1 kg per successful update, while lateral-friction ranges expand by 0.175 per update.

Evaluation Protocol. Each model is evaluated with deterministic actions for 50 episodes, using seeds 0, 42, and 99. The main metric is the final success rate, computed from `info["is_success"]` at the end of each episode. We use two target settings: an easy fixed target with the nominal target mass, and a hard target where mass and friction are randomized at test time, from a uniform distribution. We took this direction in order to test the model in various and unknow settings. This was done to achieve a single policy capable of generalizing different materials (both for the surface and the object itself) as well as different mass.

Setting	Mass (kg)	Object μ	Table μ	Object spin
Easy target	5.0	0.5	0.5	0.0
Hard target	$\mathcal{U}(5, 10)$	$\mathcal{U}(0.2, 1.5)$	$\mathcal{U}(0.2, 1.5)$	$\mathcal{U}(0, 0.01)$

Table 2. Target-domain evaluation settings used for the final success-rate comparison.

3. Experimental Results

3.1. Part 1: Hopper

Figure 1 compares REINFORCE without baseline, REINFORCE with a constant baseline of 20, and the actor-critic

variant over seeds 0, 42, and 99. All methods improve from the initial policy, but the curves remain noisy, as expected for policy-gradient training on Hopper. The constant baseline gives the strongest final training curve and reduces some early variance relative to plain REINFORCE. The actor-critic variant learns more slowly and reaches lower average returns in this implementation.

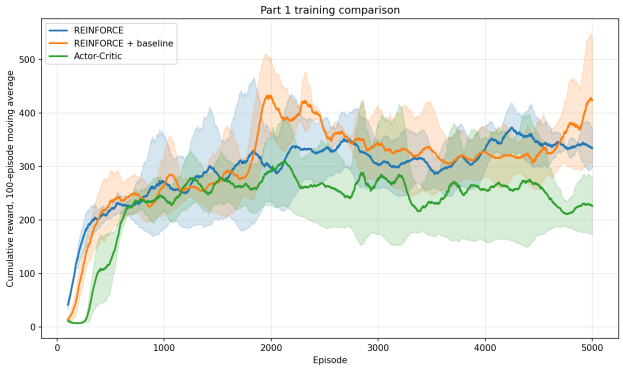

Figure 1. Part 1 training comparison on `Hopper-v4`. Curves show the 100-episode moving average of cumulative reward, averaged over seeds 0, 42, and 99; shaded regions show variability across seeds.

3.2. Nominal Sim-to-Sim Evaluation

Table 3 reports deterministic 50-episode evaluations generated from the saved SAC checkpoints. At the nominal target mass of 5 kg, the source-trained SAC policy already transfers well: the gap between source→source and source→target is small in success rate.

3.3. Harder Mass and Friction Evaluation

Because nominal mass transfer was already strong, the evaluation was made harder by randomizing parameters at test time Table 3 uses mass sampled uniformly in $[5, 10]$ kg, object/table lateral friction sampled in $[0.2, 1.5]$, and object spinning friction sampled in $[0, 0.01]$. Under this harder protocol, the non-randomized source policy drops from 97.3% to 72% success on target. The UDR policy trained with mass and friction randomization reaches 90.7%. Moreover, the ADR policy generalizes even better on unseen distribution, achieving 94% of SR on the hard evaluation setting.

3.4. UDR Hard Target vs. ADR Curriculum

An additional experiment was run to test whether training UDR directly on the target-domain intervals could outperform the ADR curriculum that starts from the source domain. The *UDR hard target* model was trained with mass sampled from $\mathcal{U}(5, 10)$ kg and lateral friction from $\mathcal{U}(0.2, 1.5)$, matching the hard evaluation distribution exactly. Intuitively, this should be the best possible UDR baseline: training and evaluation distributions coincide, so no sim-to-sim gap exists.

Table 4 reports deterministic evaluations under the hard target protocol for this model alongside the ADR source model, using 50 episodes for each seed in $\{0, 42, 99\}$.

Model	Mean return	Std return	Success
SAC UDR hard target	-6.970	16.423	85.3%
SAC ADR source	-4.638	27.440	94.0%

Table 4. Hard target deterministic evaluation aggregated over 150 episodes, using 50 episodes for each seed in $\{0, 42, 99\}$. Return mean and standard deviation are computed over all episodes. The UDR hard target model was trained with mass $\mathcal{U}(5, 10)$ kg and friction $\mathcal{U}(0.2, 1.5)$, identical to the evaluation distribution. ADR was trained starting from the source domain with an adaptive curriculum.

Despite seeing the same distribution during training and evaluation, the UDR hard target model still performs worse than the ADR curriculum model. The failure pattern reveals a friction sensitivity: the UDR policy handles low object lateral friction $\mu_{\text{obj}} \in [0.2, 0.5)$ with 100% success, but its success rate collapses to 55–73% for medium and high friction values (Table 5).

Object lateral μ bin	Success (UDR hard)	Success (ADR)
[0.21, 0.46)	100%	≈100%
[0.46, 0.72)	73%	≈100%
[0.72, 0.97)	55%	≈91%
[0.97, 1.22)	71%	≈90%
[1.22, 1.48]	64%	≈86%

Table 5. Per-friction-bin success rates on the hard target evaluation for the UDR hard target model and the ADR source model (ADR bins are approximate, from the same-seed evaluation).

4. Discussion

4.1. Result Analysis

The most important observation is that the nominal source to target transfer is easier than expected from mass alone. A source-trained SAC policy reaches 97.3% success on the 5 kg target object, only slightly below its 99.3% source success. This suggests that the current `PandaPush-v3` setup is not negatively influenced by the fixed 1 to 5 kg mass shift. The task has a fixed goal, a short object-goal distance distribution, dense rewards during training, and a powerful continuous-control algorithm. These factors can hide part of the intended reality gap. The harder setting was introduced to obtain results that are more meaningful and applicable to real-world scenarios When target mass is sampled

Model	Evaluation	Mean return	Std return	Success
SAC source	target (easy)	-1.740	7.917	97.3%
SAC target	target (easy)	-0.608	2.237	99.3%
SAC source **(Lower Bound)**	target (hard)	-12.762	19.722	72.0%
SAC target **(Upper Bound)**	target (hard)	-6.970	16.423	85.3%
SAC UDR + friction source	target (hard)	-4.814	14.555	90.7%
SAC ADR + friction source	target (hard)	-4.638	27.440	94.0%

Table 3. Deterministic SAC evaluations aggregated over 150 episodes, using 50 episodes for each seed in $\{0, 42, 99\}$. The hard target samples mass from $\mathcal{U}(5, 10)$ kg, object/table lateral friction from $\mathcal{U}(0.2, 1.5)$, and object spinning friction from $\mathcal{U}(0, 0.01)$. The hard target upper-bound checkpoint is not available yet.

in $[5, 10]$ kg and friction is randomized, the non-randomized SAC policy falls to 72.0% success. This indicates that friction and heavier masses expose failures that are not visible in the nominal target evaluation. Observing the result of the training, SAC has better performance compared to PPO.The experiments show that SAC achieves training success after 500k steps, while PPO runs are less consistent. This is coherent with the algorithms: SAC is off-policy and can reuse data through its replay buffer, whereas PPO is on-policy and discards rollouts after each update. For goal-conditioned pushing, replay-based learning is especially useful because informative transitions can be reused many times.

4.2. Why ADR Outperforms UDR Trained on the Target Distribution

The most counter-intuitive result is that the ADR source model (94.0%) outperforms the UDR hard target model (85.3%) despite the latter being trained on a distribution that exactly matches the evaluation distribution. A natural first hypothesis is that certain combinations of mass and friction are physically beyond the arm's capability: the Panda joints have limited torques (87 N on most joints), and pushing a 10 kg object against high friction may be infeasible regardless of the policy. Where is the gap coming from if both models are trained in the same environment? If some physics configurations were impossible, the ADR model would fail on them too, and the success rates would converge. Since ADR reaches 94%, the arm is capable of solving nearly all the sampled configurations; the UDR model simply has not learned how.

The more likely explanation is a curriculum effect. UDR exposes the policy to the full difficulty range from the very first episode of training. With only 500k steps and 100k warm-up steps in the replay buffer, the policy has roughly 400k gradient updates to learn a single robust strategy covering the entire $[5, 10]$ kg $\times$ $[0.2, 1.5]$ friction space simultaneously. The per-friction-bin breakdown (Table 5) shows that the UDR policy performs well at the lowest friction values and degrades at higher ones, but the mass bins do not follow a clean monotonic pattern (success is non-monotonic across the $[5, 10]$ kg range). With only 50 evaluation episodes, the per-bin sample counts are too small (6–13 episodes) to conclude that the policy systematically biased toward any particular region of the parameter space. The most robust conclusion is that the UDR policy shows high variance and inconsistent generalization across the joint mass–friction space, consistent with insufficient training under a wide uniform distribution.

ADR avoids this failure mode by construction. Starting from the nominal source dynamics, the curriculum expands the training range only when the current policy achieves a success rate above 70%. The policy is therefore never overwhelmed: it first masters easy pushes, then incrementally adapts to heavier and more friction-variable objects. By the time the curriculum reaches the hard end of the range, the policy already has a strong behavioral prior that it refines rather than learns from scratch.

This result suggests that, under a fixed compute budget, a well-designed curriculum is more data-efficient than training on the target distribution directly. It also motivates exploring ADR with friction as a first-class curriculum axis rather than expanding mass and friction simultaneously from a single success signal.

4.3. Reward Shaping

The reward-shaping wrapper affects training but not the evaluation success metric. The time penalty encourages short solutions, and the close-goal bonus increases the learning signal once the object reaches the success threshold. For SAC, this can increase success rate indirectly by making successful behavior easier to learn early in training. However, it also makes training return less comparable to raw evaluation return. This is why the evaluation script reports both return and success rate on the unwrapped environment.

4.4. Methodological Critique

The main limitation is that the current result set is not yet statistically complete. Most numbers are single-seed evaluations with 50 episodes. They are useful for debugging and

for identifying trends, but the final report should include mean and standard deviation across seeds where possible.

The domain-randomization implementation is also intentionally simple. UDR uses hand-selected uniform ranges, so its behavior depends strongly on those ranges. If the training range matches the evaluation range too closely, the comparison becomes too generous. The current setup avoids the most direct form of this issue by using conservative training friction ranges and a harder evaluation range, but a final study should report the exact train and test distributions clearly.

ADR is currently mass-only and uses a simple success-rate heuristic. It does not adapt friction, center of mass, action noise, observation noise, or target sampling. Therefore, any ADR result should be interpreted as a curriculum over object mass rather than as a full automatic domain-randomization method. In addition, the source-domain success rate may be a weak curriculum signal for transfer: a policy can be successful in the randomized training domain while still failing on the target distribution.

Finally, PPO evaluation needs care because some saved PPO models were trained behind `VecNormalize`. A valid deterministic PPO evaluation must load the corresponding normalization statistics; otherwise the policy sees observations on a scale different from training. This is marked as TODO instead of reporting unreliable numbers.

5. Conclusion

This project implemented the full path from basic policy-gradient methods on Hopper to sim-to-sim transfer experiments on PandaPush. In the current repository state, SAC is the strongest and most reliable Part 2 algorithm. At the nominal 5 kg target mass, the source-trained SAC policy already transfers well, reaching 96% success over 50 deterministic episodes, while the target-trained oracle reaches 100%. A harder mass-and-friction evaluation reveals a clearer reality gap: the non-randomized source policy drops to 76% success, while the UDR policy trained with mass and friction randomization reaches 90%. The final conclusions still require PPO, ADR, multi-seed, and Part 1 numerical evaluations to be completed.

6. Acknowledgments

6.1. Declaration of not use

The authors of this work declares that Generative AI tools were not used for writing the project report.

6.2. GenAI for the code

Generative AI tools were used for the developement of the project code as follows:

- **Name and Version**: Codex 5.5
- **Publisher**: OpenAI
- **Description**: Codex used for the following tasks: Refactoring of the repo, adding detailed readme.md, commands.md, help.md in order to let the reproducibility of experiments be smoother. Adding cli for smooth testing, parsing argument, meaningful printing for debug. Writing code for plotting
- **Name and Version**: Gemini 3.1 Pro
- **Publisher**: Google
- **Description**: Gemini was used especially for deepsearch of meaningful material to select hyperparameters before running experiments (this was done to avoid long run for tuning). The output of the search was reviewed as well as the source provided.

The authors further declares that all the logic and planning was not made using AI.

References

[1] Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. Solving rubik's cube with a robot hand. *arXiv preprint arXiv:1910.07113*, 2019. 1

[2] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pages 1861–1870, 2018. 1

[3] Sebastian Hofer, Kostas Bekris, Ankur Handa, Juan Camilo Gamboa, Melissa Mozifian, Florian Golemo, et al. Perspectives on sim2real transfer for robotics: A summary of the r:ss 2020 workshop. *arXiv preprint arXiv:2012.03806*, 2020. 1

[4] Jens Kober, J. Andrew Bagnell, and Jan Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238–1274, 2013. 1

[5] Petar Kormushev, Sylvain Calinon, and Darwin G. Caldwell. Reinforcement learning in robotics: Applications and real-world challenges. *Robotics*, 2(3):122–148, 2013. 1

[6] Fabio Muratore, Tim Gruner, Florian Wiese, Boris Belousov, Michael Gienger, and Jan Peters. Robot learning from randomized simulations: A review. *Frontiers in Robotics and AI*, 9, 2022. 1

[7] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *IEEE International Conference on Robotics and Automation*, pages 3803–3810, 2018. 1

[8] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 1

[9] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, second edition, 2018. 1

[10] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization

for transferring deep neural networks from simulation to the real world. *arXiv preprint arXiv:1703.06907*, 2017. 1